Week 19 Graded Mini Project: Fashion-MNIST Generative Modelling

Arpit Jain | Notebook: Week 19_Graded Mini Project_Jain.ipynb (all code and outputs) | Figures: results/

Objective: build four generative components on Fashion-MNIST - a Denoising Autoencoder, a VAE, a conditional GAN and a StyleGAN-lite AdaIN demo - to compare compression, sampling, conditional control and style transfer.

Task 1: Setup

1.1 Enable GPU

In Colab: Runtime → Change runtime type → GPU (T4). The notebook selects cuda, then mps (Apple-silicon GPU), then cpu. **Device used for this run: mps.**

Device check

```
import torch
device = torch.device("cuda" if torch.cuda.is_available() else "mps" if torch.backends.mps.is_available()
else "cpu")
print("device:", device)
if device.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(0))
```

1.2 Install and Import Libraries

torch, torchvision, matplotlib and tqdm are imported; missing packages are pip-installed automatically. Seeds are fixed (42) and every figure is saved to results/.

Install / import

```
# Colab already ships torch / torchvision / matplotlib / tqdm; this installs them only if missing.
import importlib.util, subprocess, sys
missing = [p for p in ["torch", "torchvision", "matplotlib", "tqdm"] if importlib.util.find_spec(p) is
None]
if missing:
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", *missing], check=True)

import json, math, random, time, warnings
warnings.filterwarnings("ignore")
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision import datasets, transforms
from torchvision.utils import make_grid
from tqdm.auto import tqdm
```

1.3 Load Fashion-MNIST

Datasets and DataLoaders (batch size 128)

```
tfm = transforms.ToTensor()                                    # pixels in [0, 1]
train_ds = datasets.FashionMNIST("data", train=True, download=True, transform=tfm)
test_ds = datasets.FashionMNIST("data", train=False, download=True, transform=tfm)
BATCH = 128
workers = 2 if device.type == "cuda" else 0
train_loader = torch.utils.data.DataLoader(train_ds, batch_size=BATCH, shuffle=True, num_workers=workers,
drop_last=True)
test_loader = torch.utils.data.DataLoader(test_ds, batch_size=BATCH, shuffle=False, num_workers=workers)
CLASSES = train_ds.classes
x0, y0 = next(iter(train_loader))
print(f"train {len(train_ds)} | test {len(test_ds)} | batch {tuple(x0.shape)} | pixel range
[{x0.min():.0f}, {x0.max():.0f}]")
print("classes:", CLASSES)
```

60,000 training and 10,000 test images, 28×28 greyscale in [0, 1], 10 clothing classes.

1.4 Create Visualisation Helper

show_grid(...)

```
def show_grid(images, nrow=8, title=None, save=None, figsize=None, ax=None):
    """Show a batch of images (N,1,28,28) in [0,1] (or [-1,1], rescaled) as one grid; optionally save
it."""
    imgs = images.detach().cpu().float()
    if imgs.min() < 0:                        # GAN outputs are in [-1, 1]
        imgs = (imgs + 1) / 2
    grid = make_grid(imgs.clamp(0, 1), nrow=nrow, padding=2, pad_value=1.0)
    own = ax is None
    if own:
```

```
        fig, ax = plt.subplots(figsize=figsize or (nrow * 0.9, math.ceil(len(imgs) / nrow) * 0.9))
    ax.imshow(grid.permute(1, 2, 0).numpy(), cmap="gray"); ax.axis("off")
    if title: ax.set_title(title)
    if own:
        plt.tight_layout()
        if save: plt.savefig(RESULTS / save, dpi=130, bbox_inches="tight")
        plt.show()
```



A training batch drawn with show_grid

Task 2: Denoising Autoencoder

2.1 Build the Model

CNN encoder (28→14→7, then linear to a **32-dim z**) and a transposed-convolution decoder (7→14→28) with a **Sigmoid** output.

Encoder / Decoder / DAE

```
LATENT_AE = 32

class Encoder(nn.Module):
    def __init__(self, z_dim, out_mult=1):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 32, 4, 2, 1), nn.ReLU(),        # 28 -> 14
            nn.Conv2d(32, 64, 4, 2, 1), nn.ReLU(),       # 14 -> 7
            nn.Flatten())
        self.fc = nn.Linear(64 * 7 * 7, z_dim * out_mult)
    def forward(self, x):
        return self.fc(self.conv(x))

class Decoder(nn.Module):
    def __init__(self, z_dim):
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(z_dim, 64 * 7 * 7), nn.ReLU())
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 4, 2, 1), nn.ReLU(),   # 7 -> 14
            nn.ConvTranspose2d(32, 1, 4, 2, 1), nn.Sigmoid())  # 14 -> 28
    def forward(self, z):
        return self.deconv(self.fc(z).view(-1, 64, 7, 7))

class DAE(nn.Module):
    def __init__(self, z_dim=LATENT_AE):
        super().__init__()
        self.enc, self.dec = Encoder(z_dim), Decoder(z_dim)
    def forward(self, x):
        return self.dec(self.enc(x))
```

2.2 Add Noise

Gaussian noise σ = 0.3, clamped to [0, 1]

```
SIGMA = 0.3
def add_noise(x, sigma=SIGMA):
    return (x + sigma * torch.randn_like(x)).clamp(0, 1)
```

2.3 Train the Model

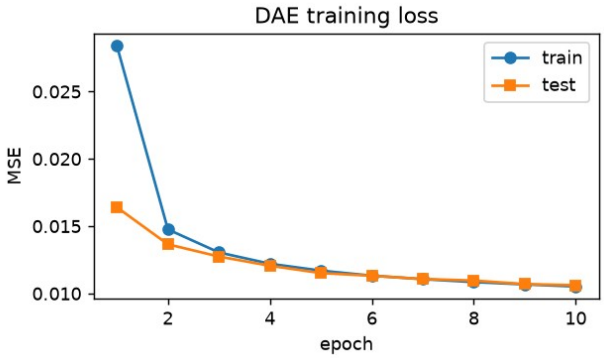
MSE between the denoised output and the clean image, Adam (lr 1e-3), 10 epochs.

Training loop

```
opt = torch.optim.Adam(dae.parameters(), lr=1e-3)
EPOCHS_DAE = 10
dae_hist = []
t0 = time.time()
for epoch in range(1, EPOCHS_DAE + 1):
```

```
    dae.train(); tr = 0
    for x, _ in tqdm(train_loader, desc=f"DAE epoch {epoch}", leave=False):
        x = x.to(device)
        loss = F.mse_loss(dae(add_noise(x)), x)
        opt.zero_grad(); loss.backward(); opt.step(); tr += loss.item()
    dae.eval(); te = 0
    with torch.no_grad():
        for x, _ in test_loader:
            x = x.to(device); te += F.mse_loss(dae(add_noise(x)), x, reduction="sum").item()
    dae_hist.append({"epoch": epoch, "train_mse": tr / len(train_loader), "test_mse": te / (len(test_ds) *
784)})
    print(f"epoch {epoch:2d} | train MSE {dae_hist[-1]['train_mse']:.4f} | test MSE {dae_hist[-1]
['test_mse']:.4f}")
```

epoch	train MSE	test MSE
1	0.0284	0.0164
2	0.0148	0.0137
3	0.0130	0.0127
4	0.0122	0.0121
5	0.0117	0.0115
6	0.0113	0.0113
7	0.0111	0.0111
8	0.0108	0.0110
9	0.0107	0.0107
10	0.0105	0.0106



2.4 Generate Outputs

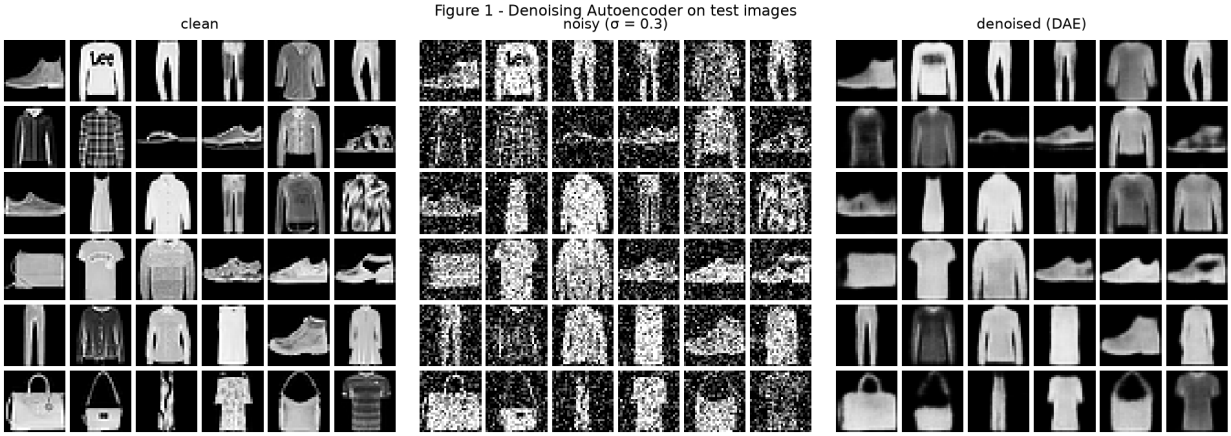
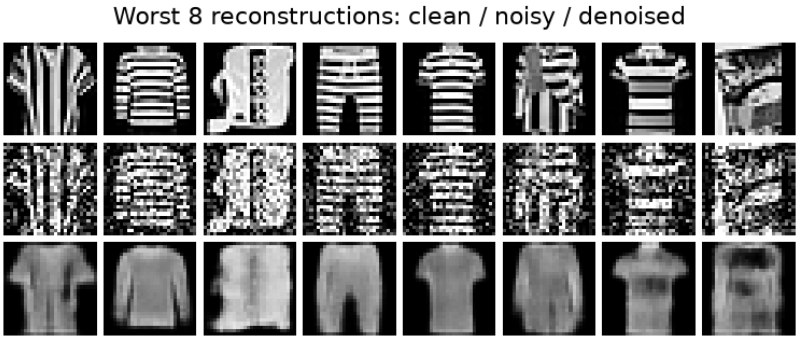


Figure 1 - clean vs noisy vs denoised (6×6 test images each)

Denoising quality: PSNR rises from 12.8 dB (noisy) to **20.5 dB** (denoised); final test MSE 0.0106. Hardest classes: Bag, Sandal, Shirt; easiest: Trouser, Sneaker, Coat.



Worst 8 test reconstructions (clean / noisy / denoised)

Observations

- **What gets fixed:** the Gaussian noise is almost completely removed. PSNR rises from 12.8 dB (noisy) to 20.5 dB (denoised), and the silhouette, the background and the overall brightness of every garment come back cleanly (Figure 1).
- **What is lost:** fine high-frequency detail. The "Lee" logo, plaid checks, stripes and printed patterns are smoothed into flat grey. A 32-number code cannot store texture, so the decoder outputs the average look of the garment's shape.
- **Failure modes:** the 8 worst reconstructions are all strongly striped or patterned tops and trousers. Their stripes disappear completely, and the noise had already hidden them at σ = 0.3.
- **Per class:** Bag (MSE 0.016), Sandal (0.015) and Shirt (0.011) are the hardest. Bags vary a lot in shape, and sandals are thin straps that the noise easily swamps. Trouser (0.006) and Sneaker are the easiest, because their shapes are simple and consistent.
- **Remaining artifacts:** outputs are slightly blurry, with soft edges and a faint halo around bright items. This is typical of an MSE-trained decoder, which averages over the possible clean images.

Task 3: Variational Autoencoder

3.1 Encoder Outputs

The same convolutional encoder outputs 2 × 16 numbers, split into **μ** and **log σ²**.

VAE: encode → (μ, log σ²), reparameterise, decode

```
LATENT_VAE = 16

class VAE(nn.Module):
    def __init__(self, z_dim=LATENT_VAE):
        super().__init__()
        self.z_dim = z_dim
        self.enc = Encoder(z_dim, out_mult=2)        # -> [mu | logvar]
        self.dec = Decoder(z_dim)
    def encode(self, x):
        mu, logvar = self.enc(x).chunk(2, dim=1)
        return mu, logvar
    def reparameterize(self, mu, logvar):
        eps = torch.randn_like(mu)                   # 3.2: z = mu + exp(0.5*logvar) * eps
        return mu + torch.exp(0.5 * logvar) * eps
    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.dec(z), mu, logvar
```

3.2 Reparameterisation

z = μ + exp(0.5 · log σ²) · ε with ε ~ N(0, I): the randomness is moved into ε, so gradients flow through μ and log σ² (see reparameterize above).

3.3 Training

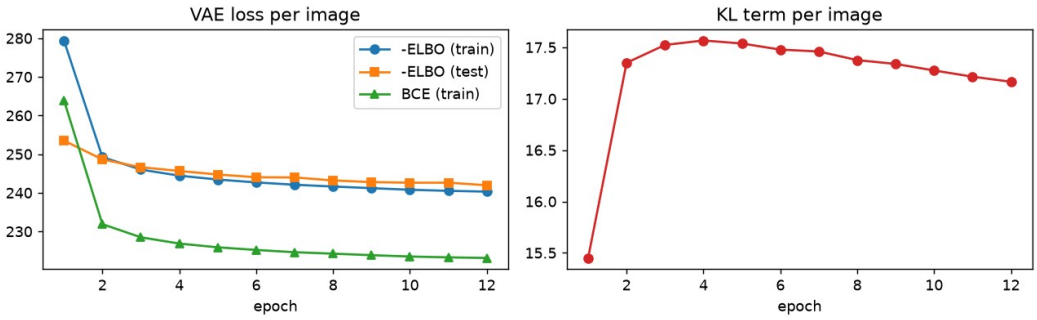
Loss = −ELBO = BCE (reconstruction, summed over pixels) + KL(q(z|x) ‖ N(0, I)), both per image; Adam (lr 2e-3), 12 epochs. **ELBO, BCE and KL are logged every epoch:**

ELBO loss

```
def vae_loss(recon, x, mu, logvar):
    bce = F.binary_cross_entropy(recon, x, reduction="sum") / x.size(0)
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp()) / x.size(0)
    return bce + kl, bce, kl
```

epoch	−ELBO (train)	BCE	KL	−ELBO (test)
1	279.41	263.96	15.45	253.50

2	249.24	231.89	17.35	248.63
3	246.01	228.49	17.52	246.62
4	244.43	226.87	17.57	245.64
5	243.43	225.89	17.54	244.74
6	242.69	225.21	17.48	244.02
7	242.09	224.63	17.46	243.97
8	241.62	224.25	17.38	243.20
9	241.20	223.86	17.34	242.77
10	240.79	223.51	17.27	242.61
11	240.51	223.30	17.21	242.62
12	240.30	223.14	17.16	241.95



3.4 Generate Samples

Figure 2 - VAE random samples (z ~ N(0, I))



Figure 2 - random samples from z ~ N(0, I)

Figure 3 - VAE latent interpolations (left: class A, right: class B)

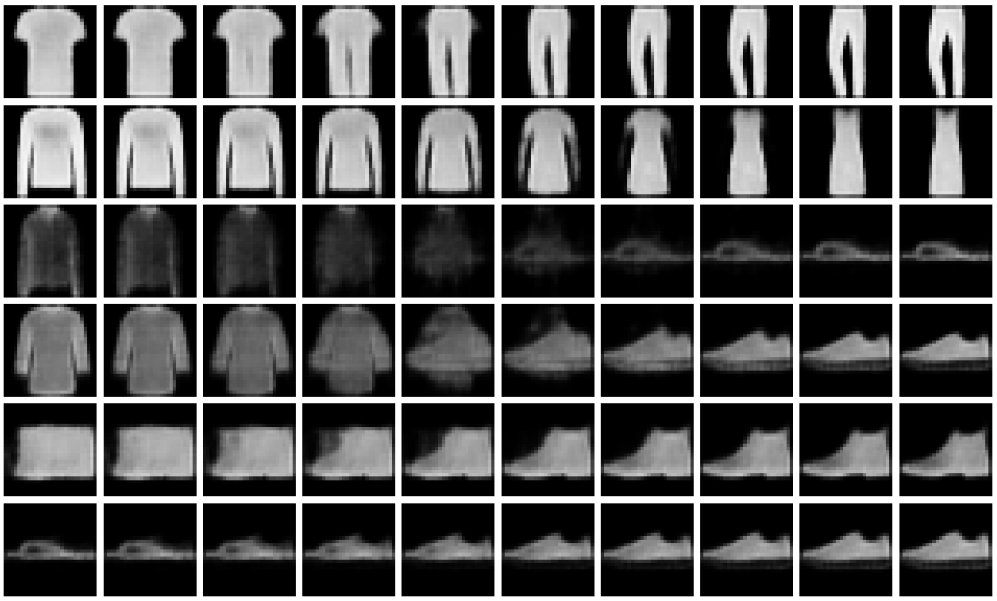


Figure 3 - interpolations between test images of two classes

Figure 4 - Traversals: each row varies one latent dim from -3 to +3 (others fixed at a T-shirt's code)

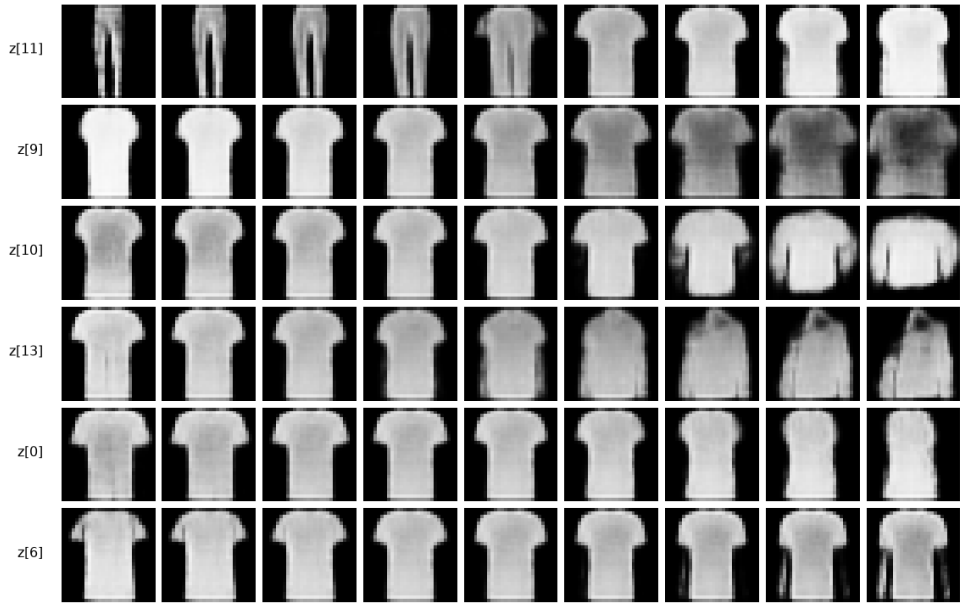


Figure 4 - traversals of the 6 highest-KL latent dims [11, 9, 10, 13, 0, 6]

Figure 4b - VAE latent means (PCA to 2-D), coloured by class

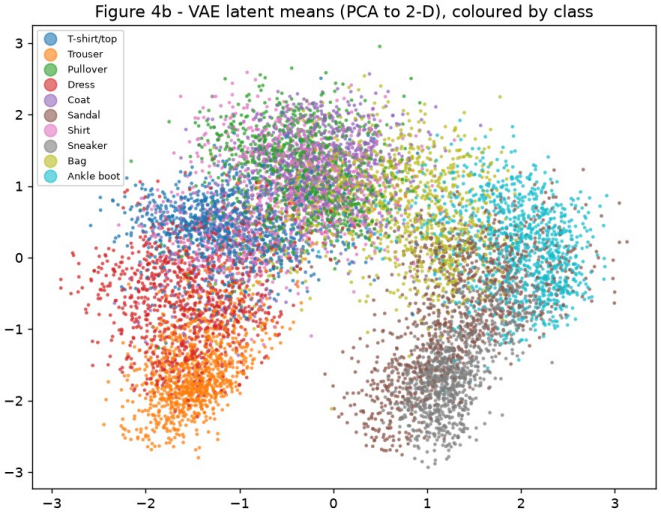


Figure 4b - latent means (PCA), coloured by class

Figure 4c - Deterministic AE vs probabilistic VAE

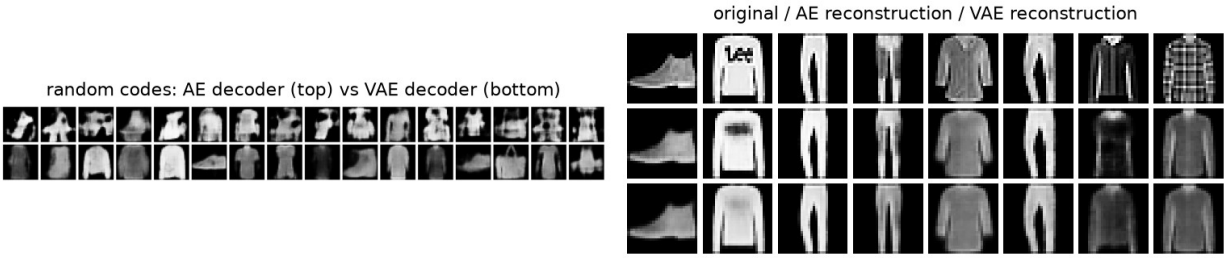


Figure 4c - same random codes through the AE and the VAE decoder; reconstructions

Traversal notes

- **z[11]** (highest KL, 2.6 nats) controls the garment type and width: at −3 the T-shirt turns into narrow trousers, and at +3 it becomes a brighter, wider top.
- **z[9]** mostly controls brightness and contrast: from a bright white shirt at −3 to a darker garment with a shaded centre at +3.
- **z[10]** adds long sleeves and widens the body, so the T-shirt turns into a pullover-like shape.
- **z[13]** introduces an asymmetric collar or front opening, so the top turns into a jacket or coat. **z[6]** gradually adds sleeve length.
- All 16 dimensions are active (KL > 0.1 nats), but the factors are entangled: one dimension changes several attributes at once (shape and brightness), as expected from a plain VAE (β = 1).

Observations: AE vs VAE

- **Sampling:** decoding random codes with the AE decoder gives broken blobs, because its latent space has gaps between the training codes. The VAE decoder turns every N(0, I) sample into a plausible garment or shoe (Figures 2 and 4c).
- **Reconstruction:** the deterministic AE reconstructs slightly better (MSE 0.0115 vs 0.0136 for the VAE's mean code). The KL term costs some detail, so VAE outputs are a little blurrier.
- **Latent structure:** the VAE's KL term organises the space. Classes form clusters, with tops, trousers, shoes and bags in distinct regions (Figure 4b), and interpolations change smoothly (Figure 3).
- **Interpolations:** within related shapes (T-shirt → trouser, pullover → dress) every step is a valid garment. Across distant classes (tops → sandals) the midpoints become faint blobs, showing low-density regions between clusters.
- **Trade-off:** the AE is a better compressor and denoiser; the VAE is a usable generative model with an ELBO of about 242 nats per image (BCE 224.6 + KL 17.3 on test).

Task 4: Conditional GAN

4.1 Implement cGAN Architecture

Generator: z (100) **concatenated with a 50-dim label embedding** → 128×7×7 → transposed convs → Tanh. Discriminator: the image stacked with a **28×28 label map** → strided convs → one logit.

Generator / Discriminator

```
Z_DIM, EMB = 100, 50

class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.emb = nn.Embedding(10, EMB)
        self.fc = nn.Sequential(nn.Linear(Z_DIM + EMB, 128 * 7 * 7), nn.BatchNorm1d(128 * 7 * 7),
nn.ReLU())
        self.net = nn.Sequential(
            nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.BatchNorm2d(64), nn.ReLU(),     # 7 -> 14
            nn.ConvTranspose2d(64, 1, 4, 2, 1), nn.Tanh())                           # 14 -> 28
    def forward(self, z, y):
        h = torch.cat([z, self.emb(y)], dim=1)                                      # concat z with label
embedding
        return self.net(self.fc(h).view(-1, 128, 7, 7))

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.label_map = nn.Embedding(10, 28 * 28)                                  # label -> 28x28 map
        self.net = nn.Sequential(
            nn.Conv2d(2, 64, 4, 2, 1), nn.LeakyReLU(0.2), nn.Dropout(0.3),          # 28 -> 14
```

```
            nn.Conv2d(64, 128, 4, 2, 1), nn.BatchNorm2d(128), nn.LeakyReLU(0.2), nn.Dropout(0.3),  # 14 ->
7
            nn.Flatten(), nn.Linear(128 * 7 * 7, 1))                                # logit
    def forward(self, x, y):
        m = self.label_map(y).view(-1, 1, 28, 28)
        return self.net(torch.cat([x, m], dim=1)).squeeze(1)
```

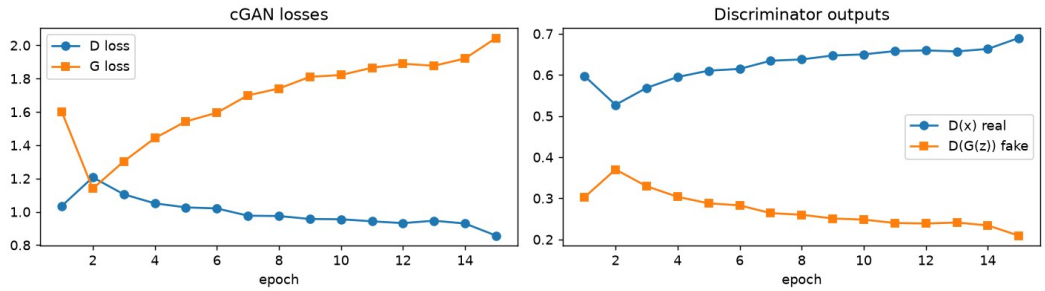
4.2 Train the cGAN

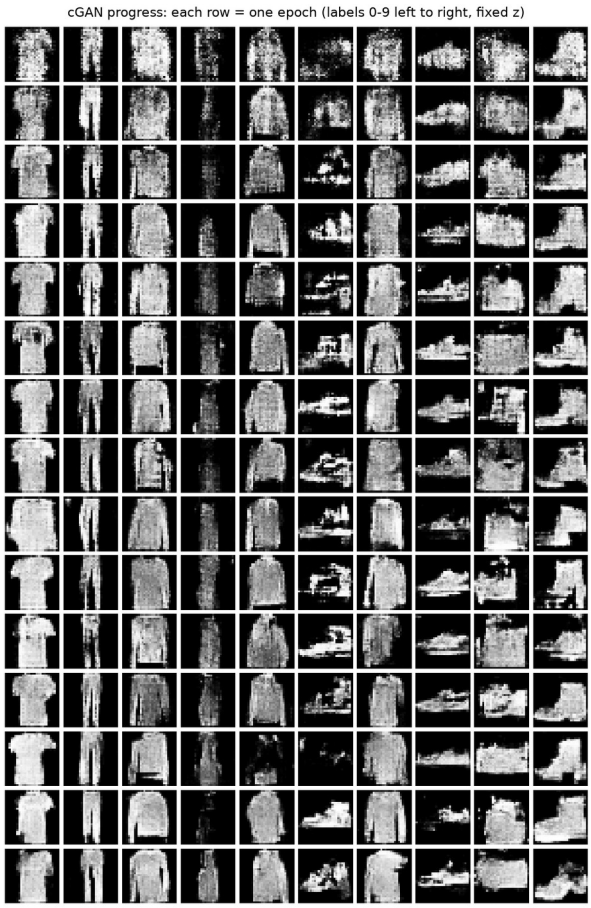
BCEWithLogits with real target 0.9 (one-sided label smoothing), Adam (2e-4, betas 0.5/0.999), 15 epochs; each step updates D on real + fake, then G with fresh labels. After every epoch labels 0-9 are drawn with a fixed z.

D and G update steps

```
for epoch in range(1, EPOCHS_GAN + 1):
    G.train(); D.train(); s = np.zeros(4)
    for x, y in tqdm(train_loader, desc=f"cGAN epoch {epoch}", leave=False):
        x, y = to_gan(x.to(device)), y.to(device); b = x.size(0)
        # --- D step: real (target 0.9) + fake (target 0)
        z = torch.randn(b, Z_DIM, device=device); fake = G(z, y)
        d_real, d_fake = D(x, y), D(fake.detach(), y)
        lossD = crit(d_real, torch.full_like(d_real, REAL)) + crit(d_fake, torch.zeros_like(d_fake))
        optD.zero_grad(); lossD.backward(); optD.step()
        # --- G step: make D call fakes real (fresh labels)
        yg = torch.randint(0, 10, (b,), device=device)
        out = D(G(torch.randn(b, Z_DIM, device=device), yg), yg)
        lossG = crit(out, torch.ones_like(out))
        optG.zero_grad(); lossG.backward(); optG.step()
        s += [lossD.item(), lossG.item(), torch.sigmoid(d_real).mean().item(),
torch.sigmoid(d_fake).mean().item()]
    s /= len(train_loader)
    gan_hist.append({"epoch": epoch, "loss_D": s[0], "loss_G": s[1], "D(x)": s[2], "D(G(z))": s[3]})
    G.eval()
    with torch.no_grad():
        epoch_grids.append(G(fixed_z, fixed_y).cpu())
    print(f"epoch {epoch:2d} | loss D {s[0]:.3f} | loss G {s[1]:.3f} | D(x) {s[2]:.2f} | D(G(z))
{s[3]:.2f}")
```

epoch	loss D	loss G	D(x)	D(G(z))
1	1.035	1.602	0.60	0.30
2	1.207	1.139	0.53	0.37
3	1.105	1.303	0.57	0.33
4	1.051	1.443	0.59	0.30
5	1.027	1.543	0.61	0.29
6	1.020	1.595	0.61	0.28
7	0.977	1.699	0.63	0.26
8	0.975	1.741	0.64	0.26
9	0.957	1.811	0.65	0.25
10	0.955	1.822	0.65	0.25
11	0.943	1.866	0.66	0.24
12	0.932	1.889	0.66	0.24
13	0.947	1.878	0.66	0.24
14	0.930	1.921	0.66	0.23
15	0.857	2.044	0.69	0.21





Per-epoch class grid (row = epoch, columns = labels 0-9, fixed z)

4.3 Generate Samples

Figure 5 - cGAN: one sample per class label

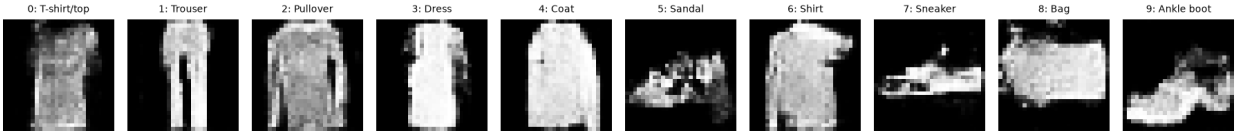


Figure 5 - one generated sample per class 0-9

Figure 6 - cGAN diversity: rows = class label 0-9, columns = different noise z

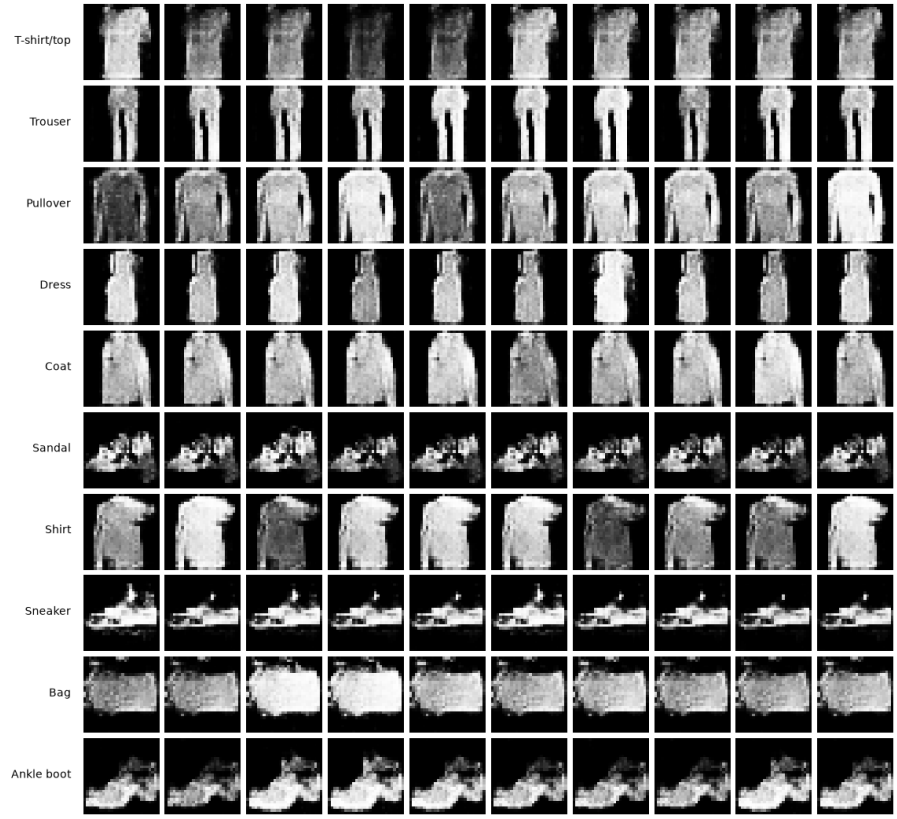


Figure 6 - diversity grid (rows = classes, columns = different z)

Conditioning check: a CNN classifier (89.3% accurate on real test images) labels 1,000 generated images; **99.8%** match the requested class (chance 10%).

class	cond. accuracy	diversity (gen)	diversity (real)
T-shirt/top	100%	4.70	8.25
Trouser	100%	1.99	6.43
Pullover	100%	4.21	9.74
Dress	100%	3.01	8.35
Coat	100%	2.29	8.95
Sandal	100%	2.31	8.42
Shirt	98%	3.91	9.61
Sneaker	100%	1.73	6.70
Bag	100%	3.08	11.27
Ankle boot	100%	3.42	8.79

Observations: conditioning behaviour

- **Labels control the class almost perfectly:** a separate CNN classifier (89.3% accurate on real images) recognises **99.8%** of 1,000 generated images as the requested class. Every class scores 100%, except Shirt at 98%, where 2 images look like pullovers (Figures 5 and 6).
- **Low within-class diversity (partial mode collapse):** different z values for the same label mostly change brightness, not shape. Average pairwise distance between samples is only 1.7-4.7, against 6.4-11.3 for real images of the same class. Sneakers, sandals and ankle boots are nearly identical across z.
- **Sharper but noisier than the VAE:** edges and dark backgrounds are crisp, and there is no blur. But textures are speckled, and sandal straps are rendered as noisy blobs, which is a typical GAN artefact at this model size and training length.
- **Training dynamics:** with the real target smoothed to 0.9, D(x) settles around 0.6-0.7 and D(G(z)) falls slowly from 0.30 to 0.21. Meanwhile the G loss rises from 1.6 to 2.0, so the discriminator is gradually winning. Training stayed stable for 15 epochs, but longer runs would need tweaks such as more dropout or a lower D learning rate.
- **The label also guides the discriminator:** the 28×28 label map lets D reject well-drawn images of the wrong class. That forces G to respect the label instead of drawing whichever class is easiest.
- **Progress grid:** class identity appears within the first 1-2 epochs. Later epochs mainly sharpen details such as collars, heels and bag handles.

Task 5: StyleGAN-lite (AdaIN Demo)

5.1 Implement StyleGAN-lite

AdaIN(c, s) = σ(s)·(c − μ(c))/σ(c) + μ(s) on the 64×7×7 feature maps of a small fully-convolutional autoencoder: content features are normalised per channel and re-scaled with the style image's channel mean/std, then decoded. 8 content and 8 style images.

Conv autoencoder + AdaIN

```
class ConvAE(nn.Module):
    def __init__(self):
        super().__init__()
        self.enc = nn.Sequential(nn.Conv2d(1, 32, 4, 2, 1), nn.ReLU(), nn.Conv2d(32, 64, 4, 2, 1),
nn.ReLU())        # 64x7x7
        self.dec = nn.Sequential(nn.ConvTranspose2d(64, 32, 4, 2, 1), nn.ReLU(), nn.ConvTranspose2d(32, 1,
4, 2, 1), nn.Sigmoid())
    def forward(self, x):
        return self.dec(self.enc(x))

def adain(content_feat, style_feat, eps=1e-5):
    c_mu, c_std = content_feat.mean((2, 3), keepdim=True), content_feat.std((2, 3), keepdim=True) + eps
    s_mu, s_std = style_feat.mean((2, 3), keepdim=True), style_feat.std((2, 3), keepdim=True) + eps
    return s_std * (content_feat - c_mu) / c_std + s_mu

cae = ConvAE().to(device); optA = torch.optim.Adam(cae.parameters(), 1e-3)
for epoch in range(1, 6):
    cae.train(); tr = 0
    for x, _ in tqdm(train_loader, desc=f"ConvAE epoch {epoch}", leave=False):
        x = x.to(device); loss = F.mse_loss(cae(x), x); optA.zero_grad(); loss.backward(); optA.step(); tr
+= loss.item()
    print(f"ConvAE epoch {epoch} | recon MSE {tr / len(train_loader):.4f}")
```

5.2 Generate Samples

Figure 7 - AdaIN style mixing: content shape + style feature statistics

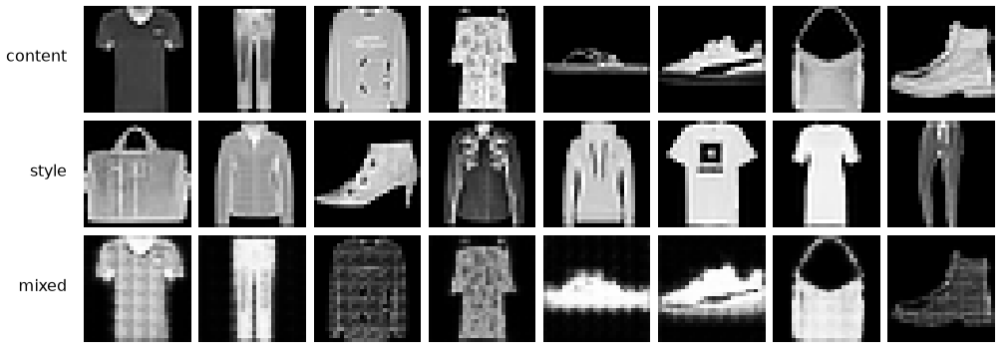


Figure 7 - content | style | mixed

style strength α = 0, 0.25, 0.5, 0.75, 1 (first pair)



Style strength α from 0 to 1 for the first pair

Observations

- **Shape comes from the content image:** the mixed image's silhouette correlates with the content (r = 0.63) far more than with the style (r = 0.15). The trousers stay trousers and the bag stays a bag.
- **Appearance comes from the style image:** AdaIN copies the style's per-channel feature statistics, and the mixed features end up much closer to the style's (mean gap 0.11) than to the content's (0.20). This shows as transferred texture (grid or plaid patterns, stripes) and brightness. Mean brightness moves from 0.25 (content) towards the style's 0.33. The α slider blends this gradually.
- **Limitation:** when the style comes from a bulky, bright garment and the content is thin (the sandal), the borrowed statistics over-activate the features and the shape blows out. This is the same content/style balance that StyleGAN controls per layer.

Analysis and Short Report

The 1-2 page analysis is provided separately (Week 19_Graded Mini Project_Jain_Analysis.pdf) and in the notebook's final section.

AE vs VAE (deterministic vs probabilistic)

- The AE maps each image to a single point and only learns to reconstruct, which gives it slightly better reconstructions (MSE 0.0115 vs 0.0136) and makes it an effective denoiser (+7.7 dB PSNR).
- The VAE maps each image to a distribution and adds a KL penalty towards N(0, I). This fills the space between codes, so random samples, interpolations and traversals decode to plausible clothes. Random AE codes decode to unusable blobs.
- The price is a little blur, and factors that are entangled rather than cleanly separated.

VAE vs cGAN (likelihood vs adversarial)

- The VAE maximises a likelihood bound (BCE + KL). That covers all the data, giving diverse samples and a smooth, meaningful latent space, but per-pixel losses average details and produce blur.
- The cGAN has no reconstruction loss. A discriminator judges realism, which gives sharper edges and darker backgrounds, but it is prone to mode collapse: within-class diversity is only about 25-55% of the real data's, and there is speckle noise.
- The VAE has an encoder and a training objective you can read and log. The GAN's losses can't be read as quality; it needs visual checks plus external metrics such as the classifier accuracy and diversity scores used here.

Why labels improve control in the cGAN

- The label is given to both networks: an embedding concatenated with z in G, and a label map in D. D therefore learns "real image of this class", and G is penalised for drawing the wrong class.
- As a result, generation is steered exactly: 99.8% of samples are recognised as the requested class. In the unconditional VAE, the class can only be found by searching the latent space.
- The label splits one hard 10-mode problem into 10 easier single-class problems. This speeds up learning (class identity appears within 1-2 epochs), but it does not prevent collapse within a class.

Takeaways: what each model captures well or poorly

- **DAE:** good at global shape, clean backgrounds and noise removal. Poor at textures, logos and thin structures, and it cannot sample.
- **VAE:** good at a smooth, sample-able latent space, interpolation and coverage of all classes. Weaker at sharpness, and its factors are entangled.
- **cGAN:** good at crisp edges and exact class control. Weaker at within-class variety and fine textures (speckle), and it needs careful monitoring.
- **AdaIN:** cleanly separates content (spatial layout) from style (channel statistics), the core idea behind StyleGAN's style control. Extreme content/style mismatches distort the shape.

Practical notes

- **Learning rates:** Adam at 1e-3 (DAE) and 2e-3 (VAE) converged within a few epochs. The VAE loss flattens after about 6 epochs. The cGAN needs the slower 2e-4 with $\beta 1 = 0.5$ to keep D and G balanced.
- **Noise:** $\sigma = 0.3$ with clamping is strong enough to hide stripes. A lower σ (0.1-0.2) would preserve more texture, and a higher one makes the DAE fall back to class-average shapes.
- **Batch size:** 128 worked for all models. BatchNorm in G and D needs batches this large, and much smaller batches make GAN training noisier.
- **Stability:** one-sided label smoothing (0.9), dropout in D and a fixed-z progress grid made the GAN trainable and easy to monitor. Signs of D gradually winning call for early stopping, or for more data augmentation / a lower D learning rate.
- **Compute:** on an Apple M1 GPU (mps) the full notebook took about 23 minutes, including 18 minutes for the cGAN. On a Colab T4 it takes about 10 minutes.